

A Laboratory Manual for

Analysis and Design of

Algorithms

(3150703)

B.E. Semester 5

(Information Technology)



Directorate of Technical Education, Gandhinagar,

Gujarat

Vishwakarma Government Engineering College
Chandkheda, Ahmedabad

Certificate

This is to certify that Mr./Ms. _____
_____ Enrollment No. _____ of B.E. Semester _____
Information Technology of this Institute (GTU Code: _____) has satisfactorily
completed the Practical / Tutorial work for the subject **Analysis and Design of**
Algorithms(3150703) for the academic year 2025-26.

Place: _____

Date: _____

Name and Sign of Faculty member

Head of the Department

Preface

Main motto of any laboratory/practical/field work is for enhancing required skills as well as creating ability amongst students to solve real time problem by developing relevant competencies in psychomotor domain. By keeping in view, GTU has designed competency focused outcome-based curriculum for engineering degree programs where sufficient weightage is given to practical work. It shows importance of enhancement of skills amongst the students and it pays attention to utilize every second of time allotted for practical amongst students, instructors and faculty members to achieve relevant outcomes by performing the experiments rather than having merely study type experiments. It is must for effective implementation of competency focused outcome-based curriculum that every practical is keenly designed to serve as a tool to develop and enhance relevant competency required by the various industry among every student. These psychomotor skills are very difficult to develop through traditional chalk and board content delivery method in the classroom. Accordingly, this lab manual is designed to focus on the industry defined relevant outcomes, rather than old practice of conducting practical to prove concept and theory.

By using this lab manual students can go through the relevant theory and procedure in advance before the actual performance which creates an interest and students can have basic idea prior to performance. This in turn enhances predetermined outcomes amongst students. Each experiment in this manual begins with competency, industry relevant skills, course outcomes as well as practical outcomes (objectives). The students will also achieve safety and necessary precautions to be taken while performing practical.

This manual also provides guidelines to faculty members to facilitate student centric lab activities through each experiment by arranging and managing necessary resources in order that the students follow the procedures with required safety and necessary precautions to achieve the outcomes. It also gives an idea that how students will be assessed by providing rubrics.

Algorithms are an integral part of computer science and play a vital role in solving complex problems efficiently. The goal of this subject is to equip students with the knowledge and skills required to design and analyze algorithms for various applications. Designing of algorithms is important before implementation of any program or solving any problem. Analysis and Design of Algorithms is essential for efficient problem-solving, optimizing resource utilization, developing new technologies, and gaining a competitive advantage. This lab manual is designed to help you learn algorithms by doing. Each experiment is structured to provide you with step-by-step instructions on how to analyze and design a particular algorithm for specific problem. You will learn how to analyze various algorithms and decide efficient algorithm in terms of time complexity. By the end of this lab, you will have a solid understanding of algorithm design and analysis.

Utmost care has been taken while preparing this lab manual however there is chances of improvement. Therefore, we welcome constructive suggestions for improvement and removal of errors if any.

Practical – Course Outcome matrix

Course Outcomes (Cos):							
1. Analyze the asymptotic performance of algorithms. 2. Derive and solve recurrences describing the performance of divide-and-conquer algorithms. 3. Find optimal solution by applying various methods. 4. Apply pattern matching algorithms to find particular pattern. 5. Differentiate polynomial and nonpolynomial problems. 6. Explain the major graph algorithms and their analyses. Employ graphs to model engineering problems, when appropriate.							
Sr. No.	Objective(s) of Experiment	CO 1	CO 2	CO 3	CO 4	CO 5	CO 6
1.	Implement a function for each of the following problems and count the number of steps executed/time taken by each function on various inputs (100 to 500) and write time complexity of each function. Also draw a comparative chart of number of input versus steps executed/time taken. In each of the following functions N will be passed by user. 1. To calculate the sum of 1 to N numbers using a loop. 2. To calculate the sum of 1 to N numbers using an equation. 3. To calculate sum of 1 to N numbers using recursion.	√					
2.	Write user defined functions for the following sorting methods and compare their performance by steps executed/time taken for execution on various inputs (1000 to 5000) of random nature, ascending order and descending order sorted data. Also draw a comparative chart of number of input versus steps executed/time taken for each cases (random, ascending, and descending). 1. Selection Sort 2. Bubble Sort 3. Insertion Sort 4. Merge Sort 5. Quick Sort	√	√				
3.	Implement a function of sequential search and count the steps executed by function on various inputs (1000 to 5000) for best case, average case and worst case. Also, write time complexity in each case and draw a comparative chart of number of input versus steps executed by sequential search for each case.	√					
4.	Compare the performance of linear search and binary search for Best case, Average case and Worst case inputs.	√	√				

5.	Implement a program for randomized version of quick sort and compare its performance with the normal version of quick sort using steps count on various inputs (1000 to 5000) of random nature, ascending order and descending order sorted data. Also draw a comparative chart of number of input versus steps executed/time taken for each cases (random, ascending, and descending).	√	√				
6.	Implement a program to solve the problem of making a change using dynamic programming.			√			
7.	Implement a program of chain matrix multiplication using dynamic programming.			√			
8.	Implement a program to solve LCS problem using dynamic programming.			√			
9.	Implement a program to solve Knapsack problem using dynamic programming.			√			
10.	Implement a program for solution of fractional Knapsack problem using greedy design technique.			√			
11.	Implement program for solution of Making Change problem using greedy design technique			√			
12.	Implement a program for Kruskal's algorithm to find minimum spanning tree.			√			√
13.	Implement a program for Prim's algorithm to find minimum spanning tree.			√			√
14.	Implement DFS and BFS graph traversal techniques and write its time complexities.						√

Industry Relevant Skills

The following industry relevant competencies are expected to be developed in the student by undertaking the practical work of this laboratory.

1. Expertise in algorithm analysis
2. Judge best algorithm among various algorithms
3. Ability to solve complex problems
4. Ability to design efficient algorithm for some problems

Guidelines for Faculty members

1. Teacher should provide the guideline with demonstration of practical to the students with all features.
2. Teacher shall explain basic concepts/theory related to the experiment to the students before starting of each practical
3. Involve all the students in performance of each experiment.
4. Teacher is expected to share the skills and competencies to be developed in the students and ensure that the respective skills and competencies are developed in the students after the completion of the experimentation.
5. Teachers should give opportunity to students for hands-on experience after the demonstration.
6. Teacher may provide additional knowledge and skills to the students even though not covered in the manual but are expected from the students by concerned industry.
7. Give practical assignment and assess the performance of students based on task assigned to check whether it is as per the instructions or not.
8. Teacher is expected to refer complete curriculum of the course and follow the guidelines for implementation.

Instructions for Students

1. Students are expected to carefully listen to all the theory classes delivered by the faculty members and understand the COs, content of the course, teaching and examination scheme, skill set to be developed etc.
2. Students shall organize the work in the group and make record of all observations.
3. Students shall develop maintenance skill as expected by industries.
4. Student shall attempt to develop related hand-on skills and build confidence.
5. Student shall develop the habits of evolving more ideas, innovations, skills etc. apart from those included in scope of manual.
6. Student shall refer technical magazines and data books.
7. Student should develop a habit of submitting the experimentation work as per the schedule and s/he should be well prepared for the same.

Common Safety Instructions

1. Switch on the PC carefully (not to use wet hands)
2. Shutdown the PC properly at the end of your Lab
3. Carefully handle the peripherals (Mouse, Keyboard, Network cable etc).
4. Use Laptop in lab after getting permission from Teacher

Index (Progressive Assessment Sheet)

Sr. No.	Objective(s) of Experiment	Page No.	Date of performance	Date of submission	Assessment Marks	Sign. of Teacher with date	Remarks
1.	Implement a function for each of following problems and count the number of steps executed/time taken by each function on various inputs (100 to 500) and write time complexity of each function. Also draw a comparative chart of number of input versus steps executed/time taken. In each of the following functions N will be passed by user. 1. To calculate the sum of 1 to N numbers using a loop. 2. To calculate the sum of 1 to N numbers using an equation. 3. To calculate sum of 1 to N numbers using recursion.						
2.	Write user defined functions for the following sorting methods and compare their performance by steps executed/time taken for execution on various inputs (1000 to 5000) of random nature, ascending order and descending order sorted data. Also draw a comparative chart of number of input versus steps executed/time taken for each cases (random, ascending, and descending). 1. Selection Sort 2. Bubble Sort 3. Insertion Sort 4. Merge Sort 5. Quick Sort						
3.	Implement a function of sequential search and count the steps executed by function on various inputs (1000 to 5000) for best case, average case and worst case. Also, write time complexity in each case and draw a comparative chart of number of input versus steps executed by sequential search for each case.						
4.	Compare the performance of linear search and binary search for Best case, Average case and Worst case inputs.						
5.	Implement a program for randomized version of quick sort and compare its performance with the normal version of quick sort using steps count on various inputs (1000 to 5000) of random						

	nature, ascending order and descending order sorted data. Also draw a comparative chart of number of input versus steps executed/time taken for each cases (random, ascending, and descending).						
6.	Implement a program to solve the problem of making a change using dynamic programming.						
7.	Implement a program of chain matrix multiplication using dynamic programming.						
8.	Implement a program to solve LCS problem using dynamic programming.						
9.	Implement a program to solve Knapsack problem using dynamic programming.						
10.	Implement a program for solution of fractional Knapsack problem using greedy design technique.						
11.	Implement a program for the solution of Making Change problem using greedy design technique.						
12.	Implement a program for Kruskal's algorithm to find minimum spanning tree.						
13.	Implement a program for Prim's algorithm to find a minimum spanning tree.						
14.	Implement DFS and BFS graph traversal techniques and write its time complexities.						
Total							

Experiment No: 1

Implement a function for each of the following problems and count the number of steps executed/time taken by each function on various inputs (100 to 500) and write equation for the growth rate of each function. Also draw a comparative chart of number of input versus steps executed/time taken. In each of the following functions N will be passed by user.

1. To calculate the sum of 1 to N numbers using a loop.
2. To calculate the sum of 1 to N numbers using an equation.
3. To calculate sum of 1 to N numbers using recursion.

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis, and Mathematical skills

Relevant CO: CO1

Objectives: (a) Compare performance of various algorithms
(b) Judge best algorithm in terms of growth rate or steps executed

Equipment/Instruments: Computer System, Any C language editor

Theory:

1. Below are the steps to calculate sum of 1 to N numbers using loop

1. Take an input value for N.
2. Initialize a variable sum to zero.
3. Start a loop that iterates from $i=1$ to $i=N$.
4. In each iteration, add the value of i to the sum variable.
5. After the loop completes, output the value of sum.

2. Below are the steps to calculate sum of 1 to N numbers using equation

1. Take an input value for N.
2. Calculate sum as $N*(N+1)/2$.
3. Output the value of sum.

3. Below are the steps to calculate sum of 1 to N numbers using recursion

1. Take an input value for N.
2. Define a recursive function $\text{sum}(n)$ that takes an integer argument n and returns the sum of 1 to n .
3. In the $\text{sum}(n)$ function, check if n equals 1. If it does, return 1 (the base case).
4. Otherwise, return the sum of n and the result of calling $\text{sum}(n-1)$.
5. Output the result.

Implement three functions based on above steps and calculate the number of steps executed by each functions on various inputs ranging from 100 to 500. Take a counter variable to calculate the number of steps and increment it for each statement in the function.

Code :

```
#include <stdio.h>
#include <conio.h>
int rec_count=0;
int sum_eq(int n){
    int count_eq=0;
    count_eq++;
    count_eq++;
    printf("Equation step count : %d\n",count_eq);
    return (n*(n+1))/2;
}
int sum_loop(int n){
    int count_loop=0;
    count_loop++;
    count_loop++;
    int i;int sum=0;
    for(i=1;i<=n;i++){
        count_loop++;
        count_loop++;
        sum+=i;
    }
    count_loop++;
    count_loop++;
    printf("Loop step count : %d\n",count_loop);
    return sum;
}
int sum_rec(int n){
    rec_count++;
    if(n==1){
        rec_count++;
        return 1;
    }
    rec_count++;
    return n+sum_rec(n-1);
}
int main(){
    int n;
    printf("Enter n: ");
    scanf("%d",&n);
    int eq_ans = sum_eq(n);
    printf("Equation answer: %d\n",eq_ans);
    int loop_ans = sum_loop(n);
    printf("Loop answer: %d\n",loop_ans);
    int rec_ans = sum_rec(n);
    printf("Recursion step count : %d\n",rec_count);
    printf("Recursion answer: %d\n",rec_ans);
    getch();
    return 0;
}
```

}

Observations:

1. Loop-Based Method:

- The number of steps increases linearly with input N.
- The step count follows the equation $2N + 2$, showing a linear time complexity $O(N)$.
- For every 100 increase in N, the steps increase by approximately 200.
- This method uses a loop to add each number from 1 to N individually.

2. Equation-Based Method:

- The number of steps remains constant (3) regardless of the input size.
- The time complexity is $O(1)$, meaning the performance does not depend on the size of input N.
- This is the fastest and most efficient method, especially for large values of N.

3. Recursion-Based Method:

- The number of steps also increases linearly with N, but slightly less than the loop method ($N + 1$ steps).
- It has $O(N)$ time complexity due to the recursive calls.
- Recursion uses stack memory and may lead to a stack overflow for very large N values. □ It is less efficient than the loop method in practice due to function call overhead.

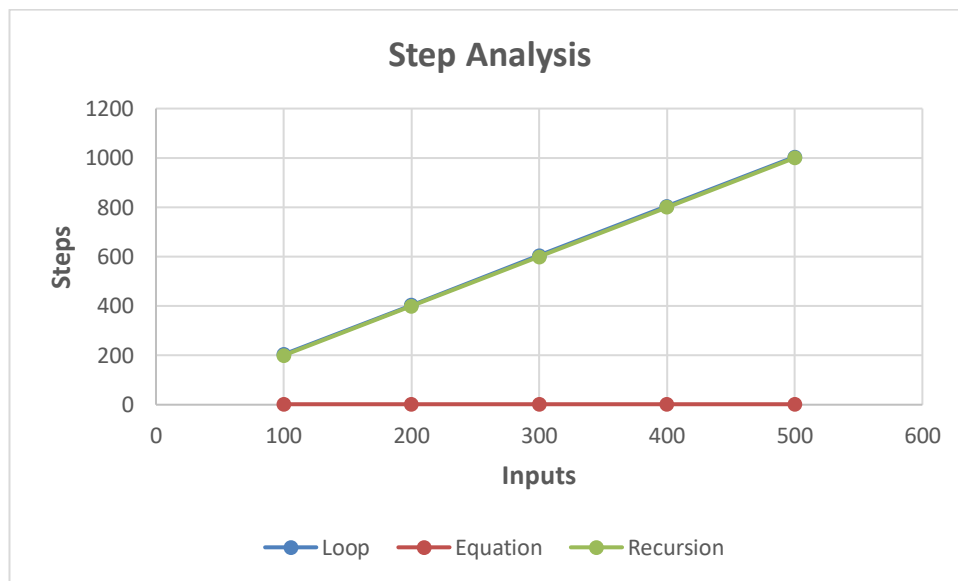
4. Comparative Analysis:

- The equation method is the most optimal in terms of steps and speed.
- Both loop and recursion methods are slower for large inputs, but recursion is generally less practical for high N due to system limitations.
- The graph clearly shows linear growth for loop and recursion, and flat (constant) growth for the equation.

Result:

Inputs	Number of Steps Executed		
	Loop method	Equations	Recursion
100	204	2	200
200	404	2	400
300	604	2	600
400	804	2	800
500	1004	2	1000
Equation	$2n+4$	2	$2n$

Chart:



Conclusion:

- For calculating sum from 1 to N, **formula method** is the most optimal.
- Both **loop** and **recursion** are less efficient as n increases, but recursion is slower due to function call overhead.

Quiz:

1. What is the meaning of constant growth rate of an algorithm?

Answer: The constant growth rate of an algorithm refers to a situation where the algorithm's running time or resource usage (like memory) does not change as the size of the input increases. In Big-O notation, this is represented as $O(1)$ - "constant time."

2. If one algorithm has a growth rate of n^2 and second algorithm has a growth rate of n then which algorithm execute faster? Why?

Answer: If one algorithm has a growth rate of n^2 (quadratic time complexity) and another has a growth rate of n (linear time complexity), the algorithm with linear time complexity (n) executes faster, especially as the input size n increases.

- Time complexity describes how the execution time of an algorithm increases with the size of the input.
- An algorithm with $O(n^2)$ means that as input size grows, the time it takes grows quadratically — if n doubles, the time increases by a factor of 4.
- An algorithm with $O(n)$ means that the time grows linearly — if n doubles, the time also doubles.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:**Rubric wise marks obtained:**

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 2

Write user defined functions for the following sorting methods and compare their performance by steps executed/time taken for execution on various inputs (1000 to 5000) of random nature, ascending order and descending order sorted data. Also, draw a comparative chart of number of inputs versus steps executed/time taken for each cases (random, ascending, and descending).

1. Selection Sort
2. Bubble Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis, and Mathematical skills

Relevant CO: CO1, CO2

Objectives:

- (a) Compare performance of various algorithms.
- (b) Judge best sorting algorithm on sorted and random inputs in terms of growth rate/time complexity.
- (c) Derive time complexity from steps count on various inputs.

Equipment/Instruments: Computer System, Any C language editor

Theory:

1. Selection Sort Function in C

```
Void SelectionSort (inta[] , int n)
//Here 'a' is the array having 'n' number of data
{
    int MinIndex,temp;
    for(int i=0;i<n-1;i++)
    {
        MinIndex=i;
        for(int j=i+1;j<n;j++)
        {
            if (a[MinIndex]>a[j])
                MinIndex=j;
        }
        if(MinIndex !=i)
        {
            temp=a[MinIndex];
            a[MinIndex] = a[j];
            a[j]=temp;
        }
    }
}
```

2. Bubble Sort Function in C

Void BubbleSort (inta[], int n)

//Here 'a' is the array having 'n' number of data

```
{
    intSwap_flag,temp;
    for(inti=0;i<n-1;i++)
    {
        Swap_flag=0;
        for(int j=0; j<n-i-1; j++)
        {
            if (a[j]>a[j+1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
                Swap_flag=1;
            }
        }
        if(swap_flag==0)
            return;
    }
}
```

3. Insertion Sort Function in C

Void InsertionSort (inta[],int n)

//Here 'a' is the array having 'n' number of data

```
{
    inti,j,key;
    for(j=1;j<n;j++)
    {
        key=a[j];
        i=j-1;
        while(i>=0 && a[i]>key)
        {
            a[i+1] = a[i];
            i= i-1;
        }
        a[i+1] =key;
    }
}
```

4. Merge Sort Function in C

void merge(inta[],int low, int mid,int high)

```
{
    int temp[5000];
    inti,j,k;
    k = low;
    j = mid +1;
    i = low;
    while((k<=mid) && (j<=high))
```

```

    {
    if(a[k]<=a[j])
        {
            temp[i] = a[k] ;
            k++;
        }
        else
        {
            temp[i] = a[j] ;
            j++;
        }
        i++;
    }
    if(k<=mid)
    for(;k<=mid;k++)
        temp[i++] = a[k];
    else
    for(;j<=high;j++)
        temp[i++] = a[j];
    for(i=low;i<=high;i++)a[i]=temp[i];
}
void merge_sort(int *a, long int low, long int high) //call this function from main
{
    if(low!=high)
    {
        int mid = (low+high)/2;
        merge_sort(a, low, mid);
        merge_sort(a, mid+1, high);
        merge(a, low, mid, high);
    }
}

```

5. Quick Sort Function in C

```

int quicksort(int q[], int lb, int ub)
{
    int flag=1;
    int i=0, j, key, t1, t2;
    if(lb<ub)
    {
        i=lb;
        j=ub+1;
        key=q[lb];

        while(flag==1)
        {
            i++;
            while(q[i]<key)
            {
                i++;
            }
            j--;
            while(q[j]>key)
            {
                j--;
            }
            if(i<j)
            {

```



```

        t1=q[i];
        q[i]=q[j];
        q[j]=t1;
    }
    else{
        flag=0;
    }
}

t2=q[lb];
q[lb]=q[j];
q[j]=t2;

quicksort(q,lb,j-1);
quicksort(q,j+1,ub);
}
return;
}

```

Write main function to use above sorting functions and calculate the number of steps executed by each functions on various inputs ranging from 1000 to 5000. Take a counter variable to calculate the number of steps and increment it for each statement in the function.

```

int main(){
    int n;
    printf("Enter n: ");
    scanf("%d",&n);
    // pass 0 - for ascending order data in generate_data function
    // pass 1 - for descending order data in generate_data function
    // pass 2 - for random order data in generate_data function
    int * a = generate_data(n,2);
    SelectionSort(a,n);
}

```

Below is the sample function that counts the number of steps:

```

voidbubble_sort(int a[], int n)
{
    intswap_flag,i,j,temp;

    steps++; //for int, it is a global variable

    for(i=0;i<n-1;i++)
    {
        steps++; //for loop
        steps++; //for swap
        swap_flag = 0;
        for(j=0;j<n-i-1;j++)
        {
            steps++; //for inner loop
            steps++; //for if
            if(a[j]>a[j+1])
            {
                temp = a[j];
                a[j] = a[j+1];
            }
        }
    }
}

```

```

        a[j+1] = temp;
        swap_flag = 1;
        steps++; //
        steps++; //
        steps++; //for four steps
        steps++; //
    }
}
steps++; //for inner loop

steps++; //for if
if(swap_flag==0)
{
    steps++; //for return
    return;
}
}
steps++; //for outer loop false
}

```

Observations:

Bubble Sort

- Performs $O(n^2)$ steps in the worst and average cases.
- For random and descending input, it performs the maximum number of steps.
- For ascending input, due to the optimization (early exit if no swaps), it performs very few steps, making it efficient for already sorted data.

Insertion Sort

- Also $O(n^2)$ in worst and average cases.
- For ascending input, it performs minimum steps (nearly $O(n)$) because no shifting is needed. □ For random or descending input, step count increases significantly due to repeated shifting.

Selection Sort

- Performs $O(n^2)$ steps regardless of input order.
- Always executes the same number of steps because it does not take advantage of already sorted data. □ Slower than Insertion Sort for nearly sorted arrays.

Merge Sort

- Performs consistently in $O(n \log n)$ steps.
- Step count is relatively stable across all input types (random, ascending, descending).
- More steps than Quick Sort for smaller arrays due to recursive calls and array copying, but efficient for large datasets.

Quick Sort

- Performs $O(n \log n)$ steps on average, but $O(n^2)$ in the worst case (e.g., already sorted or reversed if bad pivot chosen).

- For random input, performs very efficiently.
- For ascending/descending input, if not optimized with a good pivot strategy (like median or random pivot), step count increases.

Result: Complete the below table based on your implementation of functions and steps executed by each function. Also, prepare similar tables for ascending order sorted data and descending order sorted data.

For Ascending Order Data:

Inputs	Number of Steps Executed (Ascending Data)				
	Selection	Bubble	Insertion	Merge	Quick
1000	1002998	2003	4997	66026	1018984
2000	5008998	4003	9997	143062	4037984
3000	9008998	6003	14997	226834	9056984
4000	16011998	8003	19997	308134	16075984
5000	25014998	10003	24997	398234	25094984
Time Complexity	$O(n^2)$	$O(n)$	$O(n)$	$O(n \log n)$	$O(n^2)$

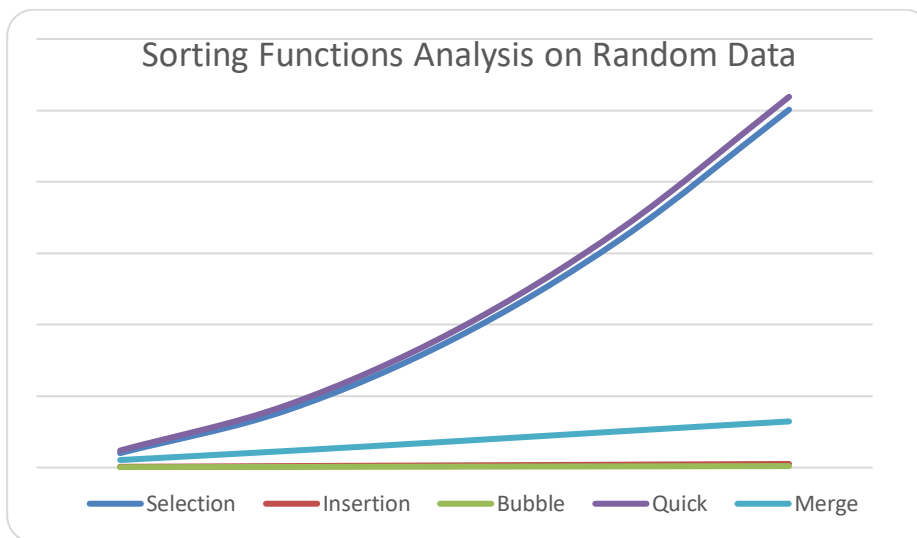
For Descending Order Data:

Inputs	Number of Steps Executed (Descending Data)				
	Selection	Bubble	Insertion	Merge	Quick
1000	1254498	3000998	1503497	65690	1018984
2000	5008998	12001998	6006997	142390	4037984
3000	11263498	27002998	13510497	223090	9056984
4000	20017998	48003998	24013997	306790	16075984
5000	31272498	75004998	37517497	391634	25094984
Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n^2)$

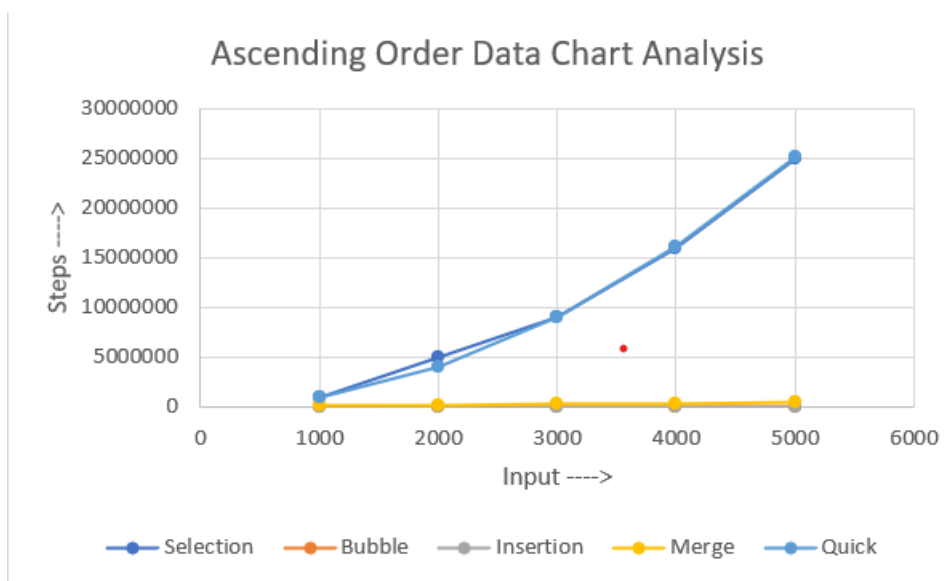
For Random Order Data:

Inputs	Number of Steps Executed (Random Data)				
	Selection	Bubble	Insertion	Merge	Quick
1000	1011500	2032479	777713	77006	44772
2000	4023787	7962369	2977651	168007	94084
3000	9037942	18139289	6869268	265285	149264
4000	16050821	32277127	12220604	364015	212626
5000	25064354	50561027	19201957	4677753	260064
Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$

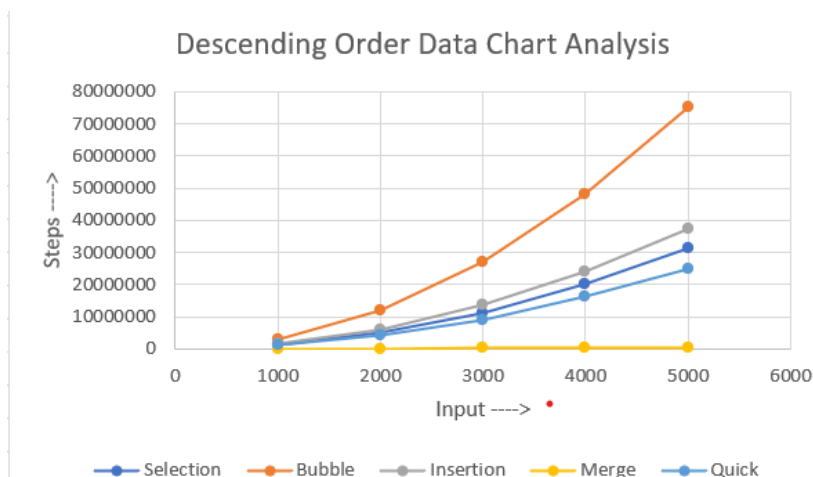
Chart:
Sample Chart is as below:



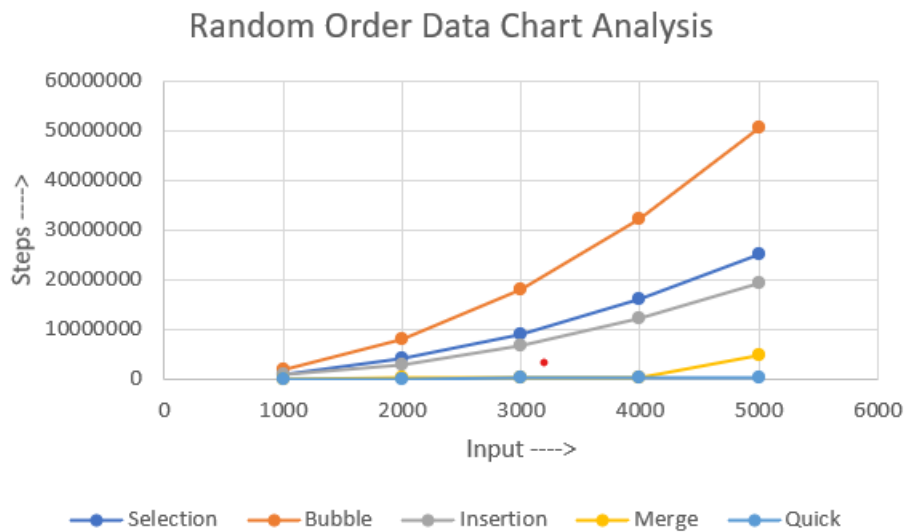
Best Case Chart Analysis:



Descending Order Data Chart Analysis:



Random Order Data Chart Analysis:



Conclusion:

- Best for Random Data: Quick Sort / Merge Sort
- Best for Sorted Data: Insertion Sort
- Worst Overall: Bubble Sort (for random/reverse data)
- Most Stable Performance: Merge Sort

Quiz:

1. Which sorting function execute faster (has small steps count) in case of ascending order sorted data?

Answer: Bubble Sort

2. Which sorting function execute faster (has small steps count) in case of descending order sorted data?

Answer: Merge Sort

3. Which sorting function execute faster (has small steps count) in case of random data?

Answer: Quick Sort

4. On what kind of data, the best case of Bubble sort occurs?

Answer: The best case of Bubble Sort occurs when the data has been sorted in Ascending order.

5. On what kind of data, the worst case of Bubble sort occurs?

Answer: The worst case of Bubble Sort occurs when the data has been sorted in Descending order.

6. On what kind of data, the best case of Quick sort occurs?

Answer: The best case of Quick Sort occurs when the data is Random order.

7. On what kind of data, the worst case of Quick sort occurs?

Answer: The worst case of Quick Sort occurs when the data has been already sorted in Ascending order.

8. Which sorting algorithms are in-place sorting algorithms?

Answer: Selection Sort, Bubble Sort, Insertion Sort and Quick Sort are in-place sorting algorithm.

9. Which sorting algorithms are stable sorting algorithms?

Answer: Bubble Sort, Insertion Sort and Merge Sort are stable sorting algorithms.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:

Rubric wise marks obtained:

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 3

Implement a function of sequential search and count the steps executed by function on various inputs (1000 to 5000) for best case, average case and worst case. Also, write time complexity in each case and draw a comparative chart of number of input versus steps executed by sequential search for each case.

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis, and Mathematical skills

Relevant CO: CO1

Objectives: (a) Identify Best, Worst and Average cases of given problem.
(b) Derive time complexity from steps count on various inputs.

Equipment/Instruments: Computer System, Any C language editor

Theory:

Steps to implement sequential search is as below:

1. Take an input array A of n elements and a key value K.
2. Define a variable pos, initially set to -1.
3. Iterate through the array A, starting from the first element and continuing until either the key value is found or the end of the array is reached.
4. For each element, compare its value to the key value K.
5. If the values match, set pos to the index of the current element and exit the loop.
6. If the end of the array is reached and the key value has not been found, pos remain equal to -1.
7. Output the value of pos.

The algorithm works by sequentially iterating through the elements of the array and comparing each element to the target value. If a match is found, the algorithm exits the loop.

Implement above functions and calculate the number of steps executed by each functions on various inputs ranging from 1000 to 5000. Take a counter variable to calculate the number of steps and increment it for each statement. Based on algorithm's logic, decide best, worst and average case inputs for the algorithm and prepare a table of steps count.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
int linearSearch(int a[], int n, int target)
{
    int count = 0;
    for (int i = 0; i < n; i++)
    {
        count++;
        count++;
        if (a[i] == target)
        {
```

```

        printf("count is : %d\n", count);
        return i;
    }
}
count++;
printf("count is : %d\n", count);
return -1;
}
int *generate_data(int n)
{
    int *p = (int *)malloc(n * sizeof(int));
    if (p == NULL)
    {
        printf("Memory not allocated.");
        exit(1);
    }
    for (int i = 0; i < n; i++)
    {
        p[i] = i;
    }
    return p;
}
int main()
{
    /*int n;
    printf("Enter size of array : ");
    scanf("%d",&n);*/
    for (int i = 1000; i <= 5000; i += 1000)
    {
        int *a = generate_data(i);
        int idx = linearSearch(a, i, a[i / 2]);
        if (idx == -1)
        {
            printf("Element not found");
        }
        else
        {
            printf("Element found at idx %d\n", idx);
        }
    }
}

```

Observations:

Random Data:

The best case takes constant time ($O(1)$), but the average case steps vary and are inconsistent. Worst case grows linearly, indicating an overall time complexity of $O(n)$ for most cases.

Ascending Data:

Best case is constant ($O(1)$), while both average and worst cases increase linearly with input size. The algorithm does not utilize the sorted order for efficiency.

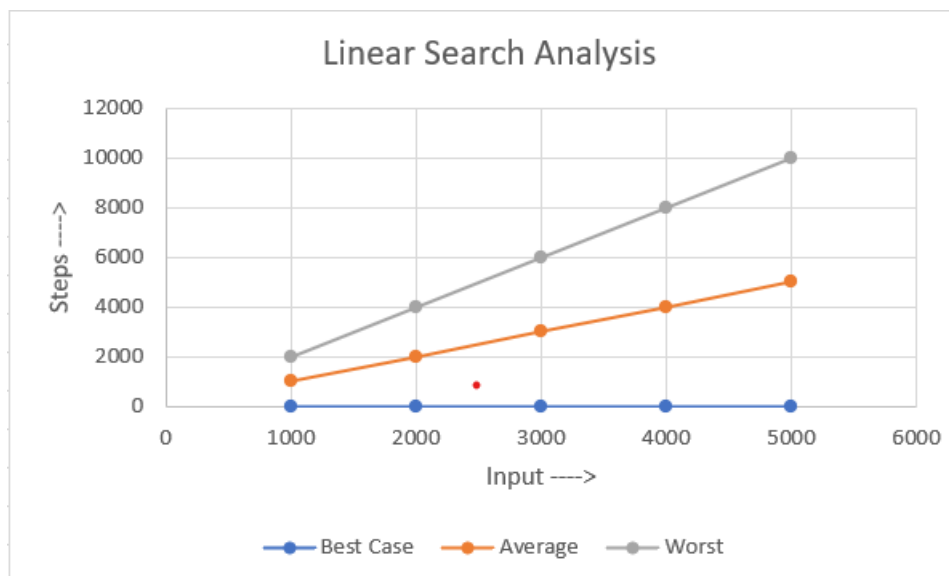
Descending Data:

Similar to ascending data, best case is constant, and average and worst cases are linear. The algorithm treats descending data the same as ascending, showing no optimization for order.

Result: Complete the below table based on your implementation of sequential search algorithm and steps executed by the function.

Inputs	Number of Steps Executed		
	Best Case	Average Case	Worst Case
1000	2	1002	2001
2000	2	2002	4001
3000	2	3002	6001
4000	2	4002	8001
5000	2	5002	10001
Time Complexity	$O(1)$	$O(n)$	$O(n)$

Chart:



Conclusion:

The algorithm has a constant-time best case across all input types, but both average and worst cases grow linearly, showing $O(n)$ time complexity. It **does not take advantage of data ordering**, performing similarly on random, ascending, and descending inputs.

Quiz:

1. Which is the best case of an algorithm?

Answer: The best case of algorithm occurs when the target element is found at 0th index and its time complexity is constant.

2. Which is the worst case of an algorithm?

Answer: The worst case of an algorithm occurs when either the element is found at last index or element is not found.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:**Rubric wise marks obtained:**

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 4

Compare the performances of linear search and binary search for Best case, Average case and Worst case inputs.

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis, and Mathematical skills

Relevant CO: CO1, CO2

Objectives: (a) Identify Best, Worst and Average cases of given problem.
(b) Derive time complexity from steps count for different inputs.

Equipment/Instruments: Computer System, Any C language editor

Theory:

Steps to implement binary search are as below:

1. Take an input **sorted** array A of n elements and a target value T.
2. Define variables start and end to represent the start and end indices of the search range, initially set to 0 and n-1 respectively.
3. Repeat the following steps while start <= end:
 - a. Calculate the midpoint index mid as $(start + end) / 2$.
 - b. If the value of the midpoint element A[mid] is equal to the target value T, return the value of mid.
 - c. If the value of the midpoint element A[mid] is greater than the target value T, set end to mid-1.
 - d. If the value of the midpoint element A[mid] is less than the target value T, set start to mid+1.
4. If the target value T is not found in the array, return -1.
5. Output the value returned in Step 3, representing the position of the target value T in the array.

Implement function of binary search algorithm and use linear search function implemented in previous practical. Compare the steps count of both the functions on various inputs ranging from 100 to 500 for each case (Best, Average, and Worst).

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
int linearSearch(int a[], int n, int target)
{
    int count = 0;
    for (int i = 0; i < n; i++)
    {
        count++;
        count++;
        if (a[i] == target)
        {
            printf("\nCount for Linear Search : %d", count);
        }
    }
}
```

```

        return i;
    }
}
count++;
printf("\nCount for Linear Search : %d", count);
return -1;
}
int binarySearch(int a[], int n, int target)
{
    int count = 0;
    int si = 0;
    int ei = n - 1;
    count++;
    while (si <= ei)
    {
        count++;
        int mid = si + (ei - si) / 2;
        count++;
        count++;
        if (a[mid] == target)
        {
            printf("\nCount for Binary Search: %d", count);
            return mid;
        }
        else if (a[mid] < target)
        {
            count++;
            si = mid + 1;
        }
        else
        {
            count++;
            ei = mid - 1;
        }
    }
    count++;
    printf("\nCount for Binary Search: %d", count);
    return -1;
}
int *generate_data(int n)
{
    int *p = (int *)malloc(n * sizeof(int));
    if (p == NULL)
    {
        printf("Memory allocation failed.");
        exit(1);
    }
    for (int i = 0; i < n; i++)
    {
        p[i] = i;
    }
    return p;
}

```

```

int main()
{
    /*int n;
    printf("Enter n:");
    scanf("%d", &n);*/
    for (int i = 100; i <= 500; i += 100)
    {
        int *a = generate_data(i);
        printf("\nCount for input : %d",i);
        printf("\nBest Case:");
        linearSearch(a, i, a[0]);
        binarySearch(a, i, a[i/2]);
        printf("\nAverage Case:");
        linearSearch(a, i, a[i/4]);
        binarySearch(a, i, a[i/4]);
        printf("\nWorst Case:");
        linearSearch(a, i, a[i]);
        binarySearch(a, i, a[i]);

    }
}

```

Observations:

Linear Search

Linear search performs the same regardless of order. If the element is at the beginning, it's fast (best case). If it's at the end or not present, it's slow (worst case).

Binary Search

Binary Search is highly efficient ($O(\log n)$) but only if the data is sorted in the correct order.

Result: Complete the below table based on your implementation of sequential search algorithm and steps executed by the function.

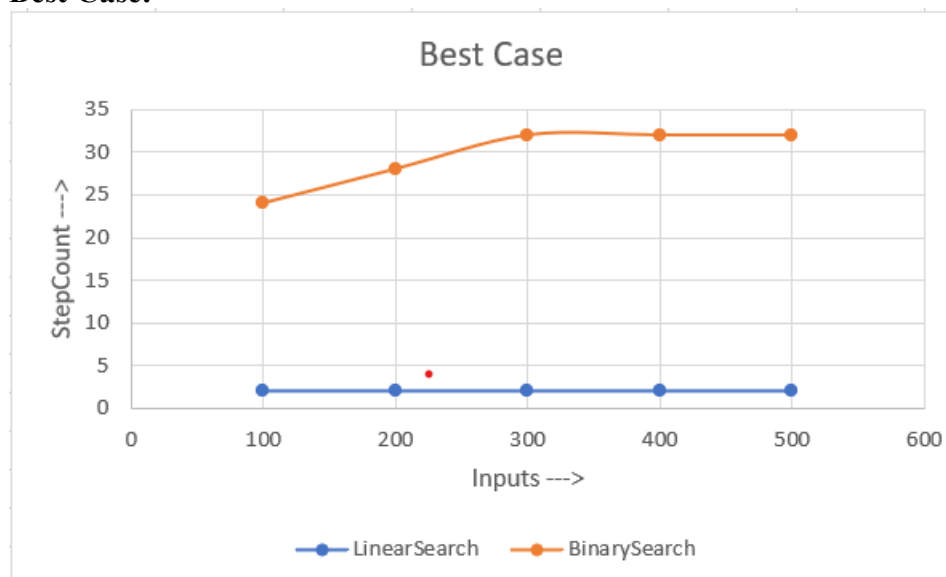
Inputs	Number of Steps Executed (Best Case)	
	Linear Search	Binary Search
100	2	24
200	2	28
300	2	32
400	2	32
500	2	32
Time Complexity	$O(1)$	$O(n \log n)$

Inputs	Number of Steps Executed (Average Case)	
	Linear Search	Binary Search
100	52	24
200	102	28
300	152	32
400	202	32
500	252	32
Time Complexity□	$O(n)$	$O(n \log n)$

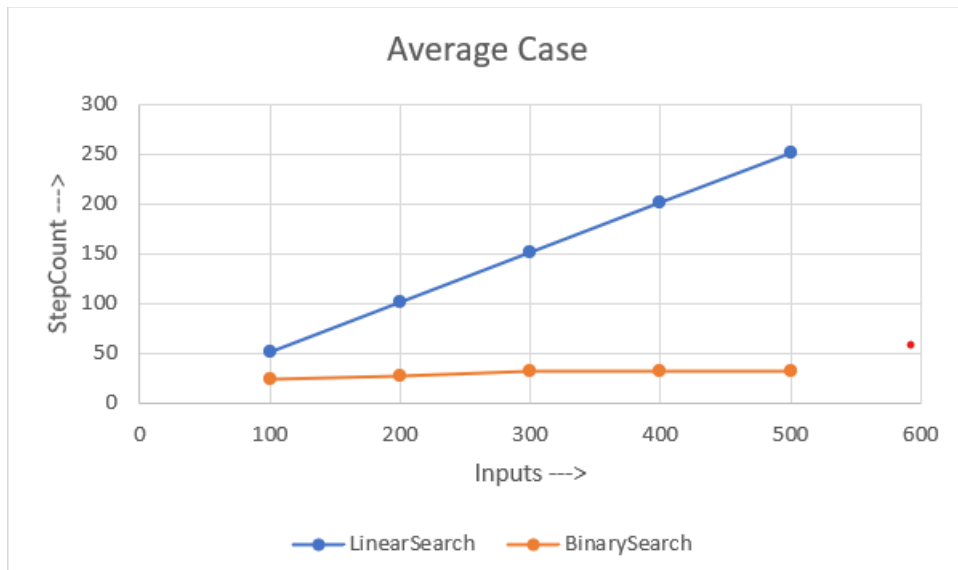
Inputs	Number of Steps Executed (Worst Case)	
	Linear Search	Binary Search
100	201	30
200	401	30
300	601	38
400	801	34
500	1001	34
Time Complexity□	$O(n)$	$O(\log n)$

Chart:

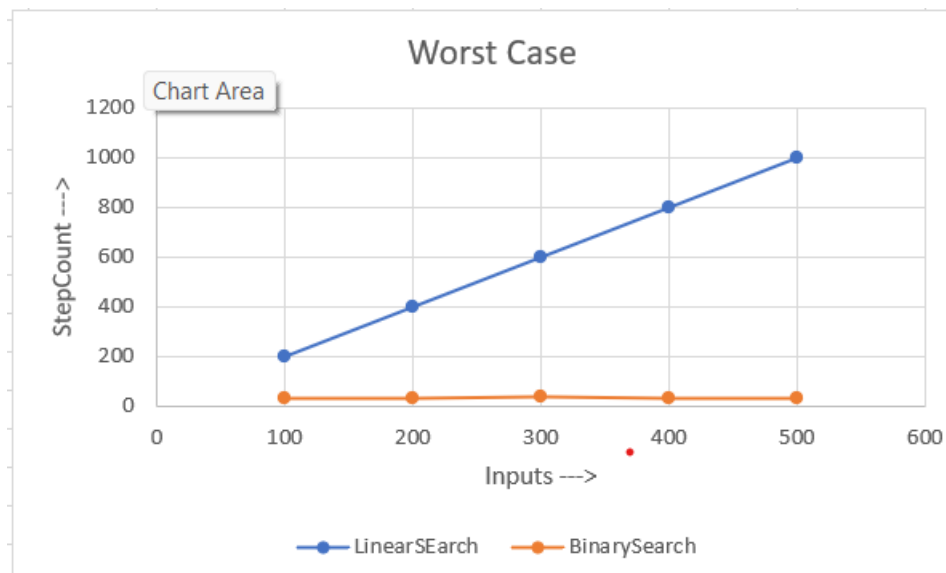
Best Case:



Average Case:



Worst Case:



Conclusion:

- **Best Case:**

Linear Search can be faster if the target is at the beginning ($O(1)$).

Binary Search takes $O(1)$ if the target is exactly at the middle, but this is rare. Linear Search may win in best case.

- **Average Case:**

Linear Search: $O(n)$ — checks about half the elements.

Binary Search: $O(\log n)$ — divides search space.

Binary Search is much faster.

- **Worst Case:**

Linear Search: $O(n)$ — checks every element.

Binary Search: $O(\log n)$ — still efficient.

Binary Search clearly better.

Quiz:

1. Which element should be searched for the best case of binary search algorithm?

Answer: The middle element should be searched for the best case of binary search algorithm.

2. Which element should be searched for the worst case of binary search algorithm?

Answer: The element which is not present in the data should be searched for the worst case of binary search.

3. Which algorithm executes faster in worst case?

Answer: Binary Search Algorithm executes faster in worst case.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:**Rubric wise marks obtained:**

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 5

Implement a program for randomized version of quick sort and compare its performance with the normal version of quick sort using steps count on various inputs (1000 to 5000) of random nature, ascending order and descending order sorted data. Also, draw a comparative chart of number of input versus steps executed/time taken for each cases (random, ascending, and descending).

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Performance analysis

Relevant CO: CO1, CO2

Objectives: (a) Improve the performance of quick sort in worst case.
(b) Compare the performance of both the version of quick sort on various inputs

Equipment/Instruments: Computer System, Any C language editor

Theory:

Steps to implement randomized version of quick sort are as below:

```
RANDOMIZED-QUICKSORT(A, low, high)
    if (low < high) {
        pivot = RANDOMIZED_PARTITION(A, low, high);
        RANDOMIZED-QUICKSORT(A, low, pivot);
        RANDOMIZED-QUICKSORT(A, pivot+1, high);
    }
RANDOMIZED_PARTITION (A, low, high) {
    pos = Random(low, high)
    pivot = A[pos]
    swap(pivot, A[low])
    left = low
    right = high
    while ( left < right ) {
        /* Move left while item < pivot */
        while( A[left] <= pivot ) left++;
        /* Move right while item > pivot */
        while( A[right] > pivot ) right--;
        if ( left < right )
            swap(A[left], A[right]);
    }
    /* right is final position for the pivot */
    swap(A[right], pivot);
    return right; }
```

Implement a function of randomized version of quick sort as per above instructions and use basic version of quick sort (that selects first element as pivot element). Compare the steps count of both the functions on various inputs ranging from 1000 to 5000 for each case (random, ascending, and descending).

Code:-

```
#include<stdio.h>
```

```

#include<stdlib.h>

int count = 0;

int quicksort(int a[], int low, int high)
{
    int flag =
    1;    count++;
        int i = 0, j, key, t1, t2;
        count++;
count++;

        if (low < high)
        {
            // Randomized pivot
selection
            int pivotIndex = low + rand() % (high - low + 1); // Random index between low and
high        t1 = a[low];    a[low] = a[pivotIndex];    a[pivotIndex] = t1;    count += 3;

            i = low;
count++;        j =
high + 1;
count++;        key
= a[low];
        count++;

            while (flag == 1)
            {
count++;
i++;
count++;
                while (a[i] < key)
                {
count++;
i++;
count++;
                }
                j--;
count++;
                while (a[j] > key)
                {
count++;
                j--;
count++;
                }
count++;
                if (i < j)
                {
                    t1 = a[i];
count++;
a[i] = a[j];
count++;
a[j] = t1;
count++;                }
            }
        }
    }
}

```

```

else      {
count++;
flag = 0;
count++;
    }
}

    t2 = a[low];
count++;
a[low] = a[j];
count++;    a[j]
= t2;
count++;

    quicksort(a, low, j - 1);
quicksort(a, j + 1, high);
    }
    count++;
return count;
}

void display(int a[], int n) {
printf("\n");   for (int i = 0;
i < n; i++) {    printf("%d
", a[i]);
    }
printf("\n");
}

void gen_ascdata(int a[], int n)
{   for (int i = 0; i < n; i++) {
a[i] = i;
    }
}

void gen_randdata(int a[], int n) {
for (int i = 0; i < n; i++) {
    a[i] = rand() % 5000;
    }
}

void gen_decdata(int a[], int n)
{   for (int i = n; i > 0; i--) {
a[n - i] = i;
    }
}

int main() {
int a[5000];
int n;

```

```
    printf("Enter no. of elements you want to sort : ");
    scanf("%d", &n);
```

```
    gen_ascdata(a, n);
    printf("Unsorted for Ascending");
    display(a, n);    quicksort(a, 0, n -
1);    printf("Count : %d\n", count);
    count = 0;
    printf("Sorted for Ascending");
    display(a, n);
```

```
    gen_randdata(a, n);
    printf("\nUnsorted for Random");
    display(a, n);    quicksort(a, 0, n -
1);    printf("Count : %d\n", count);
    count = 0;
    printf("Sorted for Random");
    display(a, n);
```

```
    gen_decdata(a, n);
    printf("\nUnsorted for
Descending");    display(a, n);
    quicksort(a, 0, n - 1);    printf("Count :
%d\n", count);    count = 0;
    printf("Sorted for Descending");
    display(a, n);
```

```
    return 0;
}
```

Observations:

Regular Quicksort

- **Ascending or Descending Data** → Very high count (close to n^2)
- **Random Data** → Moderate count ($\sim n \log n$)

Randomized Quicksort

- All data types → Consistently lower count values
- No pattern leads to worst-case behavior

Result: Complete the below table based on your implementation of sequential search algorithm and steps executed by the function.

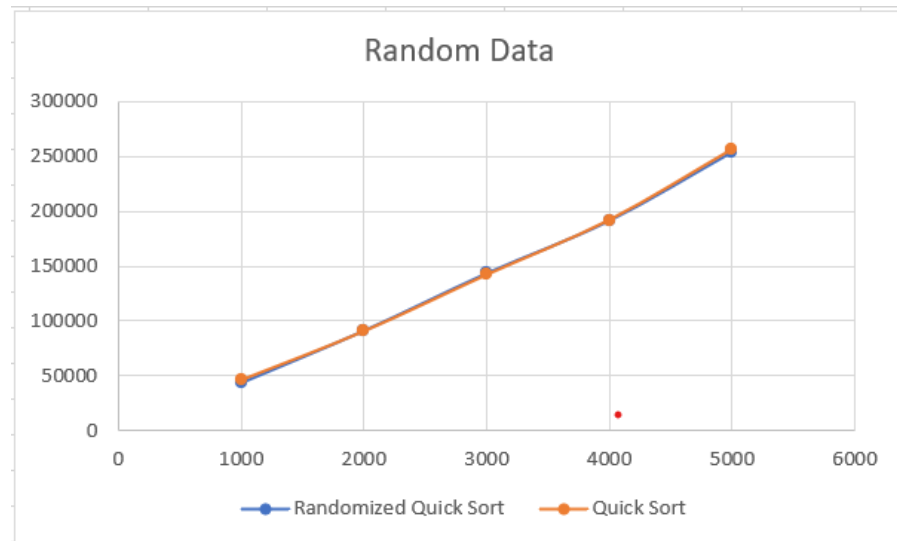
Inputs	Number of Steps Executed (Random data)	
	Randomized Quick Sort	Basic Quick Sort
1000	44315	46714
2000	91546	90950
3000	143545	142140
4000	191143	191827
5000	253248	255857
Time Complexity	$O(n \log n)$	$O(n \log n)$

Inputs	Number of Steps Executed (Ascending data)	
	Randomized Quick Sort	Basic Quick Sort
1000	37936	1018984
2000	84375	4037984
3000	123600	9056984
4000	175850	1.6E+07
5000	223128	2.5E+07
Time Complexity	$O(n \log n)$	$O(n^2)$

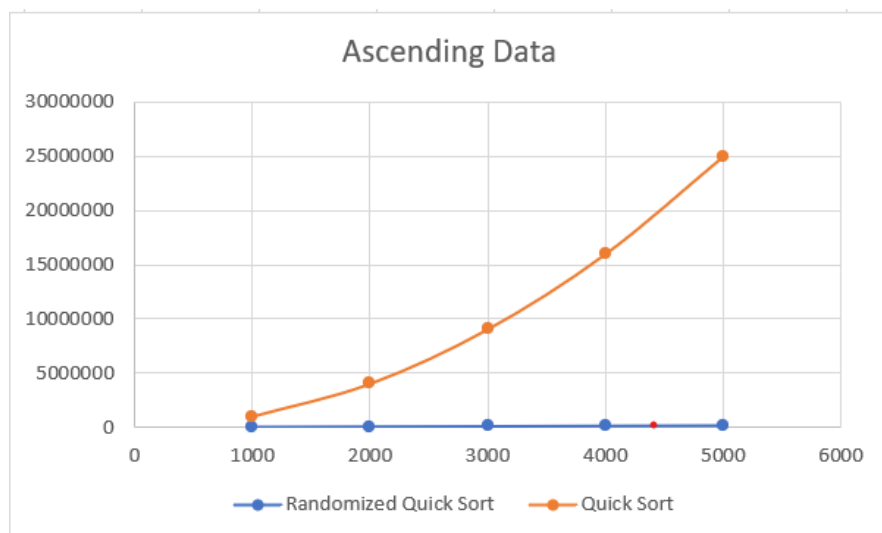
Inputs	Number of Steps Executed (Descending data)	
	Randomized Quick Sort	Basic Quick Sort
1000	43570	1023522
2000	80842	4040522
3000	126068	9057522
4000	175232	1.6E+07
5000	222144	2.5E+07
Time Complexity	$O(n \log n)$	$O(n^2)$

Chart:

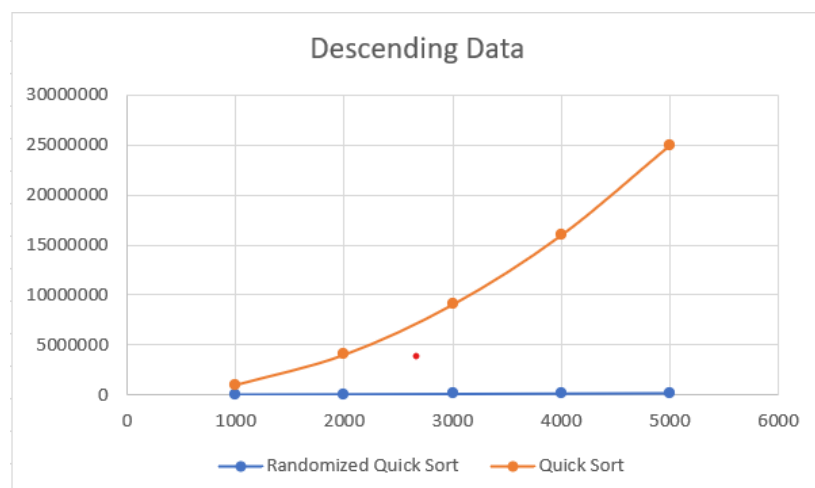
Random Data:



Ascending Order Data:



Descending Order Data:



Conclusion:

- Randomized Quick Sort is more efficient and reliable across all types of input data.
- Basic Quick Sort performs poorly on already sorted or reverse-sorted data due to unbalanced partitions.
- On sorted input, Randomized Quick Sort executes significantly faster

Quiz:

1. What is the time complexity of Randomized Quick Sort in worst case?

Answer: $O(n \log n)$

2. What is the time complexity of basic version of Quick Sort on sorted data? Give reason of your answer.

Answer: $O(n^2)$

3. Can we always ensure $O(n \lg n)$ time complexity for Randomized Quick Sort?

Answer: Yes

4. Which algorithm executes faster on ascending order sorted data?

Answer: Randomized Quick Sort

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:**Rubric wise marks obtained:**

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										

Experiment No: 10

Implement a program for solution of fractional Knapsack problem using greedy design technique.

Date:

Competency and Practical Skills: Algorithmic thinking, Programming Skills, Problem solving

Relevant CO: CO3

Objectives: (a) Understand greedy algorithm design method.
(b) Solve the optimization based problem.
(c) Find the time complexity of the algorithm.

Equipment/Instruments: Computer System, Any C language editor

Theory:

Knapsack problem is as stated below:

Given a set of items, each having different weight and value or profit associated with it. Find the set of items such that the total weight is less than or equal to a capacity of the knapsack and the total value earned is as large as possible.

Brute-force approach: The brute-force approach tries all the possible solutions with all the different fractions but it is a time-consuming approach.

Greedy approach: In Greedy approach, we calculate the ratio of profit/weight, and accordingly, we will select the item. The item with the highest ratio would be selected first.

Following are the steps to implement fractional knapsack using greedy design strategy.

1. Compute the value-to-weight ratio for each item in the knapsack.
2. Sort the items in decreasing order of value-to-weight ratio.
3. Initialize the total weight and total value to 0.
4. For each item in the sorted list:
 - a. If the entire item can be added to the knapsack without exceeding the weight capacity, add it and update the total weight and total value.
 - b. If the item cannot be added entirely, add a fraction of the item that fits into the knapsack and update the total weight and total value accordingly.
 - c. If the knapsack is full, stop the algorithm.
5. Return the total value and the set of items in the knapsack.

Implement the program based on the above logic for the solution of the fractional knapsack problem.

Observations:

Write observations based on whether this algorithm returns optimal answer or not on various inputs.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void swap_Float(float *a , float *b){  
    float temp;  
    temp = *a;  
    *a = *b;
```



```

    *b = temp;
}
void swap_Int(int *a , int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}
float KnapSack(int value[],int weight[],int n,int capacity){
    float ratio[n];
    float addedWeight = 0;
    float totalVal = 0;
    for(int i=0;i<n;i++){
        ratio[i] = (float)value[i] / weight[i];
    }
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(ratio[j]<ratio[j+1]){
                swap_Float(&ratio[j],&ratio[j+1]);
                swap_Int(&value[j], &value[j+1]);
                swap_Int(&weight[j], &weight[j+1]);
            }
        }
    }
    for(int i=0;i<n;i++){
        if(addedWeight+weight[i] <= capacity){
            addedWeight+=weight[i];
            totalVal+=value[i];
        }else{
            int remain = capacity - addedWeight;
            totalVal+=value[i]*((float)remain/weight[i]);
            break;
        }
    }
    return totalVal;
}
int main(){
    int n,capacity;
    printf("Enter number of items : ");
    scanf("%d",&n);
    int value[n];
    int weight[n];
    for(int i=0;i<n;i++){
        printf("Enter %d item weight: ",i+1);
        scanf("%d",&weight[i]);
    }
    for(int i=0;i<n;i++){
        printf("Enter %d item value: ",i+1);
        scanf("%d",&value[i]);
    }
    printf("Enter capacity of KnapScak: ");
    scanf("%d",&capacity);
    float ans = KnapSack(value,weight,n,capacity);
    printf("Total Maximum Value : %f",ans);
}

```

}

Result

```
● PS D:\ADA> gcc .\Practical10.c
● PS D:\ADA> ./a.exe
Enter number of items : 5
Enter 1 item weight: 80
Enter 2 item weight: 10
Enter 3 item weight: 40
Enter 4 item weight: 20
Enter 5 item weight: 90
Enter 1 item value: 40
Enter 2 item value: 10
Enter 3 item value: 50
Enter 4 item value: 30
Enter 5 item value: 60
Enter capacity of KnapScak: 120
Total Maximum Value : 123.333336
○ PS D:\ADA> 
```

Conclusion:

Quiz:

1. What is the time complexity of the above knapsack algorithm?

Answer: The time complexity is $O(n \log n)$ if sorting is used efficiently. In our simple implementation with nested loops, it is $O(n^2)$.

2. Does the above algorithm always return the optimal answer?

Answer: Yes, the Fractional Knapsack problem always gives the optimal answer with greedy approach, because fractions allow precise selection.

3. What is the time complexity solving knapsack problem using brute-force method?

Answer: The brute-force method checks all possible combinations, so **the time complexity is $O(2^n)$** , which is exponential and not practical for large inputs.

Suggested Reference:

1. "Introduction to Algorithms" by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein
2. "Fundamentals of Algorithms" by E. Horowitz et al.

References used by the students:

Rubric wise marks obtained:

Rubrics	Understanding of problem (3)			Program Implementation (5)			Documentation & Timely Submission (2)			Total (10)
	Good (3)	Avg. (2-1)	Poor (1-0)	Good (5-4)	Avg. (3-2)	Poor (1-0)	Good (2)	Avg. (1)	Poor (0)	
Marks										